NumPy Arrays in Two Dimensions

Shape, strides and memory layout, indexing and slicing on several axes, reductions along a named axis, reshape and length-one axes, `argmax` on more than one axis, broadcasting, and the matrix product.

01 Shape and memory layout
02 Indexing and slicing on several axes
03 Reductions along an axis
04 Reshape and length-one axes
05 argmax on several axes
06 Broadcasting
07 The matrix product
08 Loop forms and array equivalents

Scope. Arrays with a single axis are assumed. Naming an axis in a reduction, combining arrays whose shapes differ and multiplying matrices all need at least two axes.

After reading it you should be able to do the following without looking anything up.

- Predict the shape produced by an integer index and by a slice on each axis.
- Reduce along a chosen axis and state the resulting shape before running it.
- Reshape an array, insert a length-one axis, and say whether the result is a view.
- Convert between a flat index and a per-axis one.
- Apply the broadcasting rules to two shapes and predict the result, or say why it is an error.
- Use the matrix product and distinguish it from element-wise multiplication.

Every example assumes `import numpy as np`. Code comments give shapes, as `(going in) -> (coming out)`, with `v` between two operands of a ufunc. `()` is NumPy's shape for a single value.

01

Shape and memory layout

The elements of an array sit in one line in memory, whatever the shape. The shape and the strides together are the rule for turning a multidimensional index into a position in that line.

```
m = np.array([[2, 3, 4], [1, 5, 8]])   # (2, 3)

m.shape        # -> (2, 3)   two rows, three columns
m.ndim         # -> 2        number of axes
m.size         # -> 6        total elements, the product of shape
len(m)         # -> 2        length along axis 0 only, not the total
```

Axis 0 runs down the rows and axis 1 runs across the columns. For an array of three axes with shape `(2, 3, 4)` the convention continues: axis 0 selects one of the two sheets, axis 1 a row within it, axis 2 a column.

ROW-MAJOR ORDER AND STRIDES

Elements are laid out with the last axis varying fastest. A `(3, 2)` array runs `[0,0] [0,1] [1,0] [1,1] [2,0] [2,1]` through memory.

The **strides** convert an index into an address. They hold one byte count per axis: the distance in bytes to advance by one along that axis. Advancing from index `i` to `i + 1` on axis `n` adds `a.strides[n]` bytes, so `a[i, j]` sits `i * strides[0] + j * strides[1]` bytes from the start of the block.

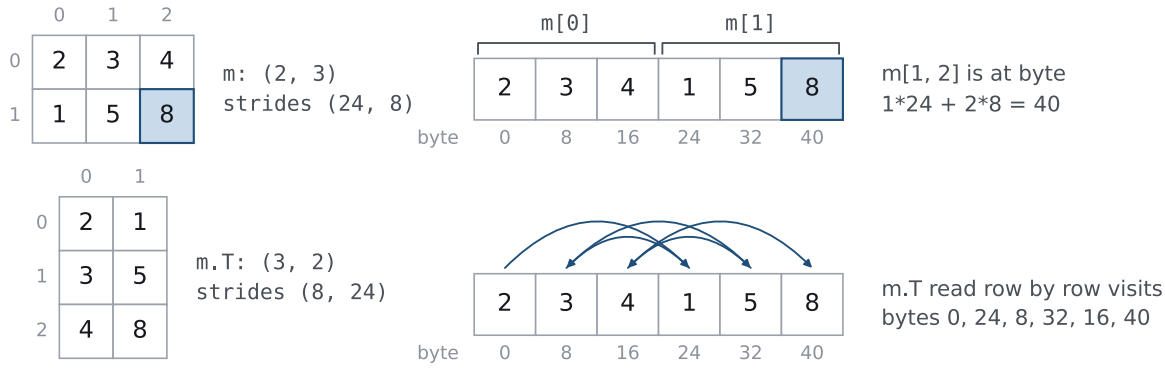
```
m = np.array([[2, 3, 4], [1, 5, 8]], dtype=np.int64)   # (2, 3) int64

m.itemsize     # -> 8         bytes per element
m.strides      # -> (24, 8)   one row is 24 bytes, one element is 8
m[1, 2]        # -> 8         at byte 1*24 + 2*8 = 40
```

Three consequences follow. Changing the shape moves nothing, because the shape is only a description of a flat block. Transposing moves nothing either: it reverses the axis order, which reverses the strides, so `m.T` is a view. A selection that cannot be reached by a constant step per axis cannot be a view.

```
# m: (2, 3) int64, strides (24, 8)

m.T                      # (2, 3) -> (3, 2)
m.T.strides              # -> (8, 24)   the same block, read the other way
m.T.base is m            # -> True      .base is the array whose memory a view uses
```



m and m.T over the same 48 bytes. m[i, j] is at byte i*24 + j*8, so m[1, 2] is at byte 40. m.T has strides (8, 24); reading it row by row visits bytes 0, 24, 8, 32, 16, 40, in the order of the arrows.

02

Indexing and slicing on several axes

Multidimensional indexing takes one entry per axis, separated by commas. An integer index and a slice differ in the dimensionality of the result.

```
b = np.array([[0, 1, 2], [3, 4, 5]])      # (2, 3)

b[1, 2]       # (2, 3) -> ()        5, an index on every axis gives one value
b[1]          # (2, 3) -> (3,)      [3 4 5], same as b[1, :]
b[1:, :]      # (2, 3) -> (1, 3)    every row but the first
b[:, 1:]      # (2, 3) -> (2, 2)    every column but the first
b[..., 0]     # (2, 3) -> (2,)      [0 3], ... fills in the remaining axes
```

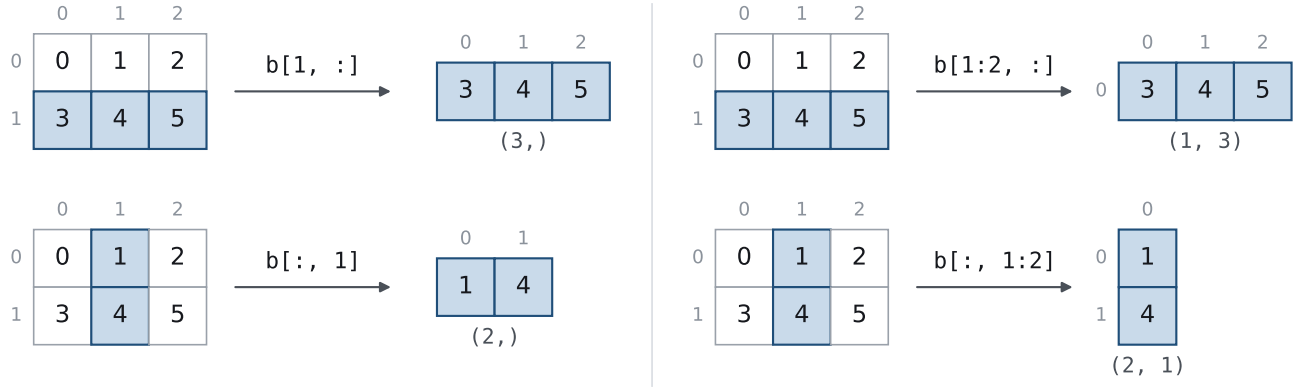
INTEGER INDEX AGAINST SLICE

An integer index removes its axis from the result and a slice keeps it, even when the slice selects exactly one element.

```
# b: (2, 3)

b[1, :]       # (2, 3) -> (3,)      axis 0 indexed away
b[1:2, :]     # (2, 3) -> (1, 3)    axis 0 sliced, so it stays, with length 1
b[:, 1]       # (2, 3) -> (2,)      axis 1 indexed away
b[:, 1:2]     # (2, 3) -> (2, 1)    axis 1 sliced, so it stays
```

The same applies at higher rank. For `x` of shape `(2, 3, 4)`, `x[:, 1]` has shape `(2, 4)` while `x[:, 1:2]` has shape `(2, 1, 4)`.



An integer index removes its axis and a slice keeps it, even when the slice has length 1. b[1, :] and b[:, 1] are one-dimensional; b[1:2, :] is a (1, 3) row and b[:, 1:2] a (2, 1) column.

A slice along any axis is still a view, exactly as on a single axis, so writing through it changes the original. An out-of-range slice is clipped. An out-of-range index raises.

B[1][2] AGAINST B[1, 2]

`b[1][2]` and `b[1, 2]` return the same value here by different steps. The first builds the intermediate row `b[1]` and then indexes it. The second goes straight to the element. The difference matters when the first index is a slice, because `b[:, 1]` and `b[:][1]` are not the same thing: `b[:]` is the whole array, so `b[:][1]` is row 1.

03

Reductions along an axis

With no argument a reduction reduces the array to a single value. The `axis` argument reduces along one axis only, and that axis is removed from the shape.

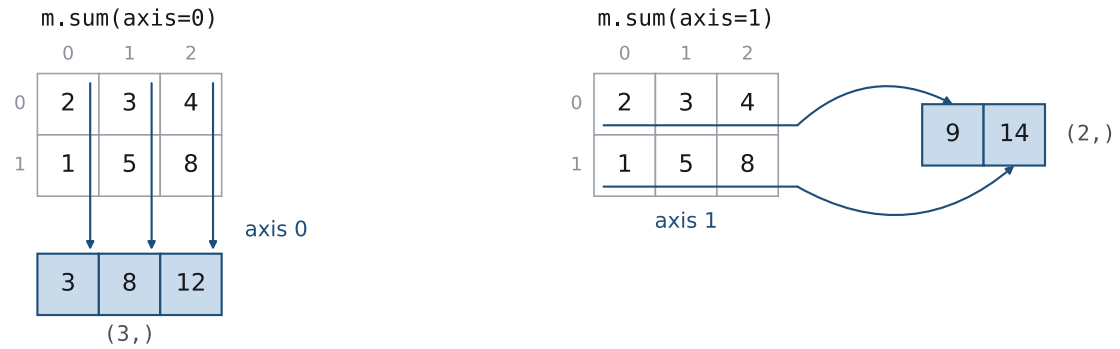
THE AXIS ARGUMENT

The NumPy documentation defines a reduction as reducing the array's dimension by one by applying the ufunc along the named axis. The named axis is therefore the one absent from the result shape. "Sum the rows" and "sum the columns" are ambiguous in English, because either phrase can name the axis that remains or the axis that is removed; the shape is unambiguous.

```
m = np.array([[2, 3, 4], [1, 5, 8]])      # (2, 3)

m.sum()              # (2, 3) -> ()      23, both axes removed
m.sum(axis=0)        # (2, 3) -> (3,)    [3 8 12], one total per column
m.sum(axis=1)        # (2, 3) -> (2,)    [9 14],   one total per row
```

For an input of shape `(2, 3)`, `axis=0` gives a result of shape `(3,)`: one value per column. The rule extends to any rank: for `x` of shape `(2, 3, 4)`, summing along axis 0, 1 and 2 gives shapes `(3, 4)`, `(2, 4)` and `(2, 3)`.



m.sum(axis=0) adds down axis 0 and leaves one total per column, shape (3,). m.sum(axis=1) adds along axis 1 and leaves one total per row, shape (2,).

Every reduction that works on a single axis also takes `axis`: `sum`, `prod`, `mean`, `min`, `max`, `std`, `var`, `any`, `all`, `argmin`, `argmax`.

KEEPDIMS

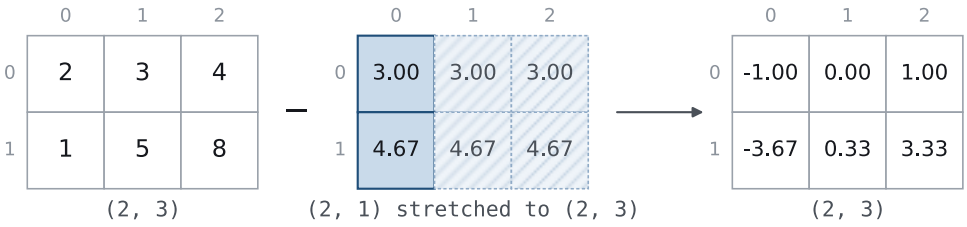
A reduction removes the axis, which makes the result awkward to combine with the original. `keepdims=True` leaves the axis in place with length 1, and that shape broadcasts back against the input.

```
# m: (2, 3)

m.sum(axis=1)                       # (2, 3) -> (2,)
m.sum(axis=1, keepdims=True)        # (2, 3) -> (2, 1)

m - m.mean(axis=1, keepdims=True)   # (2, 3) v (2, 1) -> (2, 3)   row means removed
m - m.mean(axis=1)                  # (2, 3) v (2,)   -> ValueError
```

```
m - m.mean(axis=1, keepdims=True)
```



m - m.mean(axis=1, keepdims=True): the (2, 1) column of row means is repeated across the three columns (hatched), so every row has its own mean subtracted. Values are rounded to two decimals.

04

Reshape and length-one axes

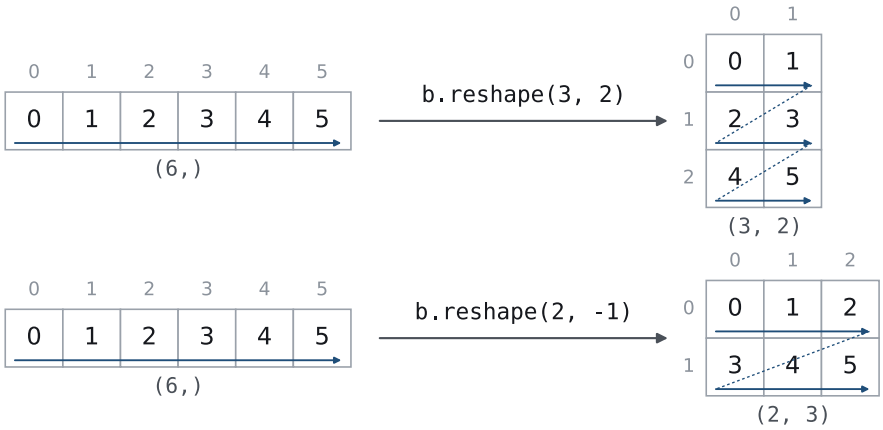
Reshaping reinterprets the same block of memory under a new index rule. The product of the new shape must equal `size`.

```
b = np.arange(6)          # (6,)  [0 1 2 3 4 5]

b.reshape(3, 2)           # (6,) -> (3, 2)   same memory
b.reshape(-1)             # any  -> (6,)     flat again, whatever went in
b.reshape(-1, 1)          # (6,) -> (6, 1)   a column
b.reshape(2, -1)          # (6,) -> (2, 3)

b.reshape(4, -1)
# ValueError: cannot reshape array of size 6 into shape (4,newaxis)
```

One axis may be written as `-1`, meaning the length that makes the total equal `size`. Only one axis may be `-1`, since two unknowns have no unique solution.



b.reshape(3, 2) and b.reshape(2, -1) read the elements of b in row-major order and fill the new shape row by row, along the drawn lines. Both results are views of b.

A reshape returns a view when the new shape can be read with one constant stride per axis over the same block, and a copy when it cannot. `np.arange(6).reshape(3, 2)` is a view, because the elements already sit in the row-major order of the new shape. `m.T.ravel()` is a copy: `m.T` has strides `(8, 24)`, so reading it row by row visits bytes 0, 24, 8, 32, 16, 40, and no single step size per axis produces that order. When an independent array is needed, `.copy()` makes one either way.

```
# m: (2, 3) int64

np.arange(6).reshape(3, 2).base is None   # -> False   a view
m.T.ravel().base is None                  # -> True    a copy
```

LENGTH-ONE AXES

Indexing with `None`, also spelled `np.newaxis`, inserts an axis of length 1 at that position. Nothing is copied and no element changes, only the shape and the strides.

```
# b: (6,)

b[None, :]                # (6,) -> (1, 6)   a row,    same as b.reshape(1, -1)
b[:, None]                # (6,) -> (6, 1)   a column, same as b.reshape(-1, 1)
```

A length-one axis fixes the orientation of a one-dimensional array combined with a two-dimensional one: `(1, 6)` is a row and `(6, 1)` a column.

05

argmax on several axes

On an array of more than one axis, `argmax` returns one index into the flattened array, not one index per axis.

```
m = np.array([[2, 3, 4], [1, 5, 8]])   # (2, 3), the maximum 8 sits at [1, 2]

m.max()                       # (2, 3) -> ()   8, the value
m.argmax()                    # (2, 3) -> ()   5, not (1, 2): the sixth in memory
m.reshape(-1)[5]              # (6,)   -> ()   8, the element at flat index 5
```

`argmax` returns a single index into the flattened array, in row-major order, with the last axis varying fastest. `np.unravel_index` converts that back into per-axis indices, and `np.ravel_multi_index` converts the other way. Both accept arrays of indices, not only single ones.

```
# m: (2, 3)

np.unravel_index(5, m.shape)                  # -> (1, 2)   one index per axis
m[np.unravel_index(m.argmax(), m.shape)]      # -> ()       8
```

With an `axis` argument, `argmax` returns one index per row or per column. As in any reduction, the named axis is removed from the result shape.

```
# m: (2, 3)

m.argmax(axis=1)              # (2, 3) -> (2,)   [2 2],   column of each row's maximum
m.argmax(axis=0)              # (2, 3) -> (3,)   [0 1 1], row of each column's maximum
```

ARGMAX ON A SLICE

Taking `argmax` of `grid[:, c0:c1]` gives a flat index into the slice, whose width is `c1 - c0`, not into `grid`. Converting it back needs both the slice's own shape and its offset:

```
grid = np.arange(240 * 360).reshape(240, 360)   # (240, 360)
c0, c1 = 100, 140                               # a band of 40 columns

band = grid[:, c0:c1]                     # (240, 360) -> (240, 40)
r, c = np.unravel_index(band.argmax(), band.shape)   # -> one index per axis of band
row, col = r, c + c0                      # c was measured from the band's left edge
```

Ties are broken by position: the first occurrence in memory is returned.

06

Broadcasting

A ufunc requires its operands to have the same shape. Broadcasting is the set of rules under which unequal shapes are treated as equal, by repeating elements virtually rather than copying them.

THE BROADCASTING RULES

Two steps, applied in this order:

1. **Padding with leading ones.** If the two shapes have a different number of axes, `1` is prepended to the shorter one until both have the same length. The NumPy documentation states this in two places: missing dimensions are assumed to have size one, and a shape of `(6,)` "acts like a `(1, 6)` array". The ones go on the **left**, so the two shapes line up from the right.
2. **Position-by-position comparison.** Two lengths are compatible when they are equal, or when one of them is `1`. The result takes the larger of the two, so a length of 1 stretches to match the other. If any position is neither, the operation raises `ValueError`.

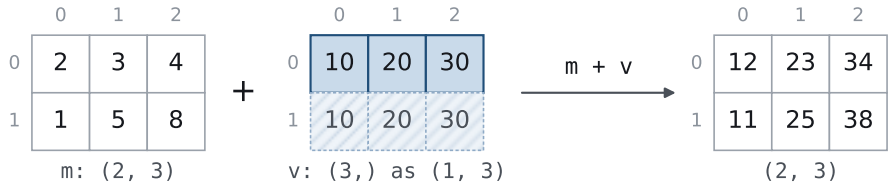
Three cases, with the padding step written out:

```
a       (3d)    2 x 3 x 4
b       (1d)            4   padded to    1 x 1 x 4
result  (3d)    2 x 3 x 4                 2 v 1 -> 2,  3 v 1 -> 3,  4 v 4 -> 4

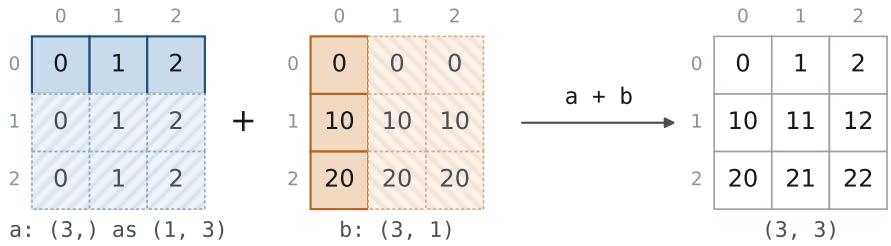
a       (1d)            3   padded to        1 x 3
b       (2d)        3 x 1
result  (2d)        3 x 3                 1 v 3 -> 3,  3 v 1 -> 3,  both stretch

a       (2d)        2 x 3   same length, nothing to pad
b       (2d)        3 x 2
error                                     3 v 2 fails, and 2 v 3 fails
```

```
v = np.array([10, 20, 30])
```




```
a = np.array([0, 1, 2])          b = np.array([[0], [10], [20]])
```



Top: v of shape (3,) is padded to (1, 3) and repeated down the two rows of m. Bottom: a, padded to (1, 3), and b, of shape (3, 1), both stretch, and the result is (3, 3). Hatched cells are repeated without being copied.

In the middle case neither operand is a subset of the other: `(3,)` pads to `(1, 3)`, every position holds a `1` against a `3`, so both sides stretch and the result is larger than either input. That is the shape of an outer product.

The same rules on more shape pairs:

```
given                     padded                      result
(240, 360)    (360,)      (240, 360)    (1, 360)    -> (240, 360)
(240, 360)    (240, 1)    (240, 360)    (240, 1)    -> (240, 360)
(15, 3, 5)    (3, 1)      (15, 3, 5)    (1, 3, 1)   -> (15, 3, 5)
(8, 1, 6, 1)  (7, 1, 5)   (8, 1, 6, 1)  (1,7,1,5)   -> (8, 7, 6, 5)
(2, 1)        (8, 4, 3)   (1, 2, 1)     (8, 4, 3)   -> error, 2 against 4
```

```
np.zeros((2, 3)) + np.zeros((3, 2))
# ValueError: operands could not be broadcast together with shapes (2,3) (3,2)
```

Step 1 is also why a one-dimensional operand acts per column by default. A `(360,)` vector against a `(240, 360)` grid pads to `(1, 360)`, which is one value per **column**, applied to every row. One value per **row** needs the padding on the other side, written explicitly: `[:, None]` turns `(240,)` into `(240, 1)`. `np.newaxis` is the documented spelling of that `None`.

```
grid = np.zeros((240, 360))      # (240, 360)

per_column = np.zeros(360)       # (360,)   one value per column
grid - per_column                # (240, 360) v (360,)   -> (240, 360)

per_row = np.zeros(240)          # (240,)   one value per row
grid - per_row[:, None]          # (240, 360) v (240, 1) -> (240, 360)
```

MEMORY COST OF THE RESULT

No copy is made of the stretched operand, so broadcasting itself allocates nothing. The *result* is a real array, and it has the broadcast shape. Comparing every element of an array against every other is a case where the result is large:

```
a = np.arange(10_000)                  # (10_000,)

a[:, None] > a[None, :]                # (10_000,1) v (1,10_000) -> (10_000, 10_000)
                                       # of bool
(a[:, None] > a[None, :]).sum(axis=1)  # -> (10_000,)   after 100 million booleans
```

That expression replaces a double loop, at the cost of allocating the full `n` by `n` result: 100 million booleans here, and 10 GB at `n = 100_000`. A sort plus a binary search computes the same counts in `O(n log n)` time and allocates `n` integers.

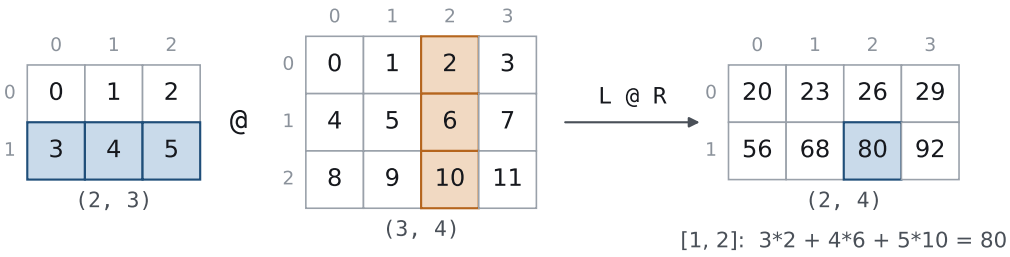
07

The matrix product

A different operation from element-wise multiplication, with a different shape rule.

```
L = np.arange(6).reshape(2, 3)       # (2, 3)
R = np.arange(12).reshape(3, 4)      # (3, 4)

L @ R                # (2, 3) @ (3, 4) -> (2, 4)   np.matmul(L, R), or L.dot(R)
L * R                # (2, 3) v (3, 4) -> ValueError, not broadcastable
```



L @ R: entry [1, 2] of the (2, 4) result is row 1 of L against column 2 of R, 3*2 + 4*6 + 5*10 = 80. The inner lengths, 3 and 3, have to agree.

For `(m, k) @ (k, n)` the inner dimensions must agree and the result is `(m, n)`. Entry `[i, j]` is the sum over the shared axis of row `i` of the left operand against column `j` of the right. The order of the operands matters: `L @ R` and `R @ L` are different operations, and usually only one of them is defined.

When the shapes do not line up, transposing one operand often removes the error but computes a different product from the one intended.

`@` also handles the one-dimensional cases. A `(n,)` against a `(n,)` is the inner product and gives a scalar, and a `(m, n)` against a `(n,)` gives `(m,)`.

08

Loop forms and array equivalents

Common nested-loop forms and their array equivalents. Each expression in the right-hand column runs in compiled code and references no index variable.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Total of each row	`arr.sum(axis=1)`
Total of each column	`arr.sum(axis=0)`
The largest row total	`arr.sum(axis=1).max()`
Sum inside a rectangle	`grid[r0:r1, c0:c1].sum()`
Subtract each row's mean from that row	`arr - arr.mean(axis=1, keepdims=True)`
Subtract one value per column	`grid - per_column`
Subtract one value per row	`grid - per_row[:, None]`
Column of the maximum in each row	`arr.argmax(axis=1)`
Position of the overall maximum	`np.unravel_index(arr.argmax(), arr.shape)`
Compare every element against every other	`a[:, None] > a[None, :]`
Matrix product	`L @ R`

FURTHER READING

- Broadcasting, the rules with more worked shape pairs.
- Indexing on ndarrays. The basic indexing section covers indexing and slicing on several axes and `np.newaxis`.
- Linear algebra, for what exists beyond the matrix product itself.
- The routine reference, every NumPy function grouped by topic.